XIAMEN UNIVERSITY MALAYSIA
厦門大學馬來西亞分校

EEE312 Week5

Lab Practice

C Program to Compute the Eigenvalue and Eigenvector

Write a C source code to solve any system of 4 linear equations in the form of Ax=b using the Gaussian elimination method. Your program should display the matrices U and v at the end of the row operations and the values of the variables x at the end of the back substitutions.

.

// C Source code

```
1   // gaussianelimination.c
2   // Solve linear equation Ax = b using Gaussian elimination method
3   #include <stdio.h>
4
5   #define N 4
6
7   void rowoperation(double A[][N], double b[N]); // function to perform row operation on A and b
8   void backSubstitution(double U[][N], double v[N], double x[N]); // perform back substitution on U and v
9   void printMatrix(double A[][N]); // function to print out matrix A
10  void printVector(double b[]); // function to print out vector b
```

Line 7: function rowoperation() to perform row operation on augmented matrix A | b.

Line 8: function backSubstitution() to perform back substitution on augmented matrix U | v.

Line 9: function printMatrix() to print the elements of matrix A or U.

Line 10: function printVector() to print the elements of vector b, v or x.

Function printMatrix()

```
67  void printMatrix(double A[][N])
68  {
69      for(int i=0; i<N; i++)
70          for(int j=0; j<N; j++)
71          {
72              printf("%.4f\t\t", A[i][j]);
73              if(j==N-1) printf("\n");
74          }
75      printf("\n");
76  }
```

Line 72: Print the current element A[i][j] of matrix A.

Line 73: Print a newline character if the end of the row I is reached.

Function printVector ()

```
78  void printVector(double b[])
79  {
80      for(int i = 0; i<N; i++)
81      {
82          printf("%.4f\n", b[i]);
83      }
84      printf("\n");
85  }
```

Line 82: Print the current element b[i]of vector b.

Function rowOperation ()

Line 36-46: Perform row operation on matrix A.

Line 47: Perform row operation on element iof vector busing row operation b[i] ← b[i] ← m*b[k].

```c
void rowoperation(double A[][N], double b[])
{
    // row operations
    double m;
    for(int k=0; k<N-1; k++) // do for column k=0 to k=N-2;
    {                        // k is the row index where the pivot element is located
        for(int i=k+1; i<N; i++) // do for column below k from k+1 to N-1;
        {                        // i is the row index where there is NO pivot element
            m = A[i][k]/A[k][k]; // A[k][k] is the pivot
            for(int j=0; j<N; j++) // For current row i, perform element wise operation from 0 to N-1.
            {                      // j is the column index
                A[i][j] -= m*A[k][j]; // At row i, perform A[i][j]<-A[i][j]-m*A[k][j] on matrix A

            }
            b[i] -= m*b[k]; // For element i of vector b, calculate b[i]<-b[i]-m*b[k]
        }
    }
}
```

Function backSubstidtution()

Line 61 & 63: Implement the equation below to obtain x[i].

$$x[i] = \left[v[i] - \sum_{j=i+1}^{j<N} (U[i][j] \cdot x[j]) \right] \Big/ U[i][i]$$

```c
void backSubstitution( double U[][N], double v[], double x[])
{
    double m = 0;

    for(int i=N-1; i>=0; i--) // for every row index i=N-1 to i=0
    {
        m = 0.0;
        for(int j=N-1; j>i; j--) // for every column index j=N-1 to j>i
        {
            m += U[i][j]*x[j];// calculate the sum of U[i][j]*x[j]
        }
        x[i] = (v[i]-m)/U[i][i]; // calculate x[i] using the standard equation
    }
}
```

Function main()

```c
int main()
{
    double A[N][N] = {{2.0,-1,7,-1},{-2,5,-3,-8},{8,2,-1,2},{1,-8,1,-2}}; //  coefficient matrix A
    double b[N] = {-1, 3, -2, 4}; //  matrix b
    double x[N] = {0.0}; // unknown variables

    printf("The original matrix A is \n");
    printMatrix(A); // print the original matrix A
    printf("The original matrix b is \n");
    printVector(b); // print the original matrix b
    rowoperation(A, b); // forward row operation
    backSubstitution(A, b, x); // back substitution operation
    printf("The matrix U after the Gaussian elimination is\n");
    printMatrix(A); // print the matrix U after row operation
    printf("The matrix v after the Gaussian elimination is\n");
    printVector(b); // print the matrix v after row operation
    printf("Solutions to linear equations are: \n");
    printVector(x); // print the solved variables
    return 0;
}
```

Output

```
The original matrix A is
2.0000          -1.0000         7.0000          -1.0000
-2.0000         5.0000          -3.0000         -8.0000
3.0000          2.0000          -1.0000         2.0000
1.0000          -8.0000         1.0000          -2.0000

The original matrix b is
-1.0000
3.0000
-2.0000
4.0000

The matrix U after the Gaussian elimination is
2.0000          -1.0000         7.0000          -1.0000
0.0000          4.0000          4.0000          -9.0000
0.0000          0.0000          -35.0000                19.5000
0.0000          0.0000          0.0000          -15.5893

The matrix v after the Gaussian elimination is
-1.0000
2.0000
-1.0000
8.1071

Solutions to linear equations are:
-0.0504
-0.4089
-0.2612
-0.5200
```